

```
print('Enter the letters of "from" and "to" towers, or QUIT.')
print("(e.g., AB to moves a disk from tower A to tower B.)")
print()
response = input("> ").upper().strip()
```

---

We start an infinite loop that continues looping until either a `return` statement causes the execution to leave the loop and function or a `sys.exit()` call terminates the program. The first part of the loop asks the player to enter a move by specifying *from* and *to* towers.

Notice the `input("> ").upper().strip()` instruction that receives keyboard input from the player. The `input("> ")` call accepts text input from the player by presenting a `>` prompt. This symbol indicates that the player should enter something. If the program didn't present a prompt, the player might momentarily think the program had frozen.

We call the `upper()` method on the string returned from `input()` so it returns an uppercase form of the string. This allows the player to enter either uppercase or lowercase tower labels, such as 'a' or 'A' for tower A. In turn, the uppercase string's `strip()` method is called, returning a string without any whitespace on either side in case the user accidentally added a space when entering their move. This user friendliness makes our program slightly easier for players to use.

Still in the `getPlayerMove()` function, we check the input the user enters:

---

```
if response == "QUIT":
    print("Thanks for playing!")
    sys.exit()

# Make sure the user entered valid tower letters:
if response not in ("AB", "AC", "BA", "BC", "CA", "CB"):
    print("Enter one of AB, AC, BA, BC, CA, or CB.")
    continue # Ask player again for their move.
```

---

If the user enters 'QUIT' (in any case, or even with spaces at the beginning or end of the string, due to the calls to `upper()` and `strip()`), the program terminates. We could have made `getPlayerMove()` return 'QUIT' to indicate to the caller that it should call `sys.exit()`, rather than have `getPlayerMove()` call `sys.exit()`. But this would complicate the return value of `getPlayerMove()`: it would return either a tuple of two strings (for the player's move) or a single